

README

Helix Constitution

Field	Value
Revision	4
Created	2026-05-14
Last modified	2026-07-23T10:55:33Z
Status	active
Status summary	Universal engineering constitution shared by every project that includes this repository as a Git submodule. R4 refresh (2026-07-23 — the §11.4.226/§11.4.227 reopen-root-cause-harvest round; prior R3 refresh = review Minor M3): the anchor corpus now extends through §11.4.227 (§11.4.226 evidence-class-at-closure + standing detection pressure; §11.4.227 governance-corpus self-custody; §11.4.225 scheduler-quota burst-throttling telemetry) plus the §12.11/§12.12 host-safety axes, with governance mirrored in lockstep across CLAUDE.md / AGENTS.md / QWEN.md / GEMINI.md (§11.4.157); the authoritative per-round history lives in Constitution.md's own header Status summary. The previous R2 blurb (Recent additions 2026-05-20: §11.4.73–78 ...) had been stale for roughly 150 anchors.
Issues	none
Issues summary	—
Fixed	initial creation (R1, 2026-05-14); README overhaul + recent-additions refresh (R2, 2026-05-20); revision-header + recent-anchor-coverage refresh (R3, 2026-07-23 — review Minor M3 of commit 698dd52).
Fixed summary	—

Field	Value
Con- tinu- ation	—

The universal engineering constitution shared by every project that includes this repository as a Git submodule.

What this is

This repository contains the **universal, project-agnostic** rules, constraints, and conventions that every Helix project (and any other project that opts in) inherits automatically. Project-specific rules live in the consuming project's own `Constitution.md` / `CLAUDE.md` / `AGENTS.md` / `QWEN.md` and **extend** what is in here.

The model is three-layer (top-to-bottom evaluation, project overrides universal when conflicts are explicit):

1. **Base layer (this submodule)** — universal rules: anti-bluff covenant (§11.4), data safety, host safety, test coverage, commit/ push discipline, credentials handling (§11.4.10), documentation discipline (§11.4.65), submodule-catalogue-first discovery (§11.4.74), containers-submodule mandate (§11.4.76).
2. **Project layer** — project root `Constitution.md` / `CLAUDE.md` / `AGENTS.md` / `QWEN.md` : domain-specific rules, hardware-specific constraints, service-specific configuration.
3. **Subdirectory layer (optional)** — `<subdir>/CLAUDE.md` for module-local overrides. CLI agents (Claude Code, Codex, Cursor, OpenCode, Qwen Code, Kimi CLI, Crush) merge these by walking up from the working directory.

How to consume

. Add the submodule

In each consuming project:

```
git submodule add git@github.com:HelixDevelopment/HelixConstitution.git constitution

# Pin to a tag for reproducibility (recommended)
cd constitution
git checkout v1.0.0      # whatever the current stable tag is
cd ..
git add constitution
git commit -m "chore: add Helix Constitution submodule pinned to v1.0.0"
```

When teammates clone your project:

```
git clone --recurse-submodules <your-project-url>
# or if already cloned:
git submodule update --init --recursive
```

. Wire the inheritance

Add a clearly-marked pointer at the top of your project's root `CLAUDE.md` :

```
## INHERITED FROM constitution/CLAUDE.md

All rules in `constitution/CLAUDE.md` (and the `constitution/Constitution.md`
it references) apply unconditionally. Project-specific rules below
extend them.

@constitution/CLAUDE.md
```

Same for `AGENTS.md` (any AI-agent tooling) and (since 2026-05-20) `QWEN.md` for Qwen Code:

```
> Base agent rules: `constitution/AGENTS.md` — READ IT FIRST.
> The base file is authoritative for any topic not covered here.
```

And for your project Constitution:

This constitution **extends** the Helix Universal Constitution at ``constitution/Constitution.md``. All clauses there apply unless
³ explicitly overridden below with an explicit ``Override §X.Y`` section.

. Set up the multi-upstream push

From the **constitution submodule's** root directory:

```
cd constitution
bash install_upstreams.sh
```

This reads every `Upstreams/*.sh` declaration and configures matching git remotes locally, so a single `git push` reaches every provider (GitHub, GitLab, GitFlic, GitVerse).

Alternatively, if your toolkit ships the system-wide `install_upstreams` command (⁴Project-Toolkit `Upstreamable/`), you can invoke that instead — it reads the same `Upstreams/` directory.

. Verify inheritance with an automated test

Every consuming project should ship a test that verifies the constitution submodule is present, at the expected pinned revision, and that the project's `CLAUDE.md` / `AGENTS.md` / `QWEN.md` / `Constitution` all reference the submodule. See:

- The ATMOSphere project's `test_constitution_inheritance.sh` for the reference implementation pattern.
- The Herald project's `tests/test_constitution_inheritance.sh` + paired `tests/test_i6_refinement_meta.sh` for an example with refined I6 invariant (allows non-constitution submodules) per the original pattern.
- The Herald `scripts/audit_antibluff.sh` for the canonical anti-bluff audit pattern (3 invariants: §11.4 anchor across submodules + default test suite green + paired §1.1 meta-test green).

Contents

File	Purpose
<code>Constitution.md</code>	The canonical universal constitution. All clauses are project-agnostic.
<code>CLAUDE.md</code>	Universal CLAUDE.md for Claude Code agents. Imports <code>Constitution.md</code> by reference.
<code>AGENTS.md</code>	Universal AGENTS.md for every other CLI agent (Codex, Cursor, Aider, OpenCode, Crush, Kimi CLI).
<code>QWEN.md</code>	Universal QWEN.md for the Qwen Code CLI agent. Created 2026-05-20 per the §11.4.76 propagation set.
<code>submodules-catalogue.md</code>	Canonical inventory of every owned-by-us repository under <code>vasic-digital</code> + <code>HelixDevelopment</code> (142 repos at landing time). Created 2026-05-20 per §11.4.74 follow-up mandate.
<code>LICENSE</code>	License governing the constitution text itself.
<code>Upstreams/</code>	One <code>.sh</code> declaration file per upstream Git remote.
<code>install_upstreams.sh</code>	Configures every declared upstream as a local git remote.
<code>find_constitution.sh</code>	Helper that locates this submodule from arbitrary nested depth.
<code>meta_test_inheritance.sh</code>	Sentinel-based meta-test that verifies the inheritance gate catches a deletion of the §11.4 anchor.
<code>templates/</code>	Templates for project-specific Constitution / CLAUDE / AGENTS / QWEN files.

Recent additions (— —)

The following clauses + companion files landed this session in response to operator mandates. Each entry includes the verbatim user mandate in its forensic-anchor block within `Constitution.md` :

Clause	Topic
§11.4.73	Main-specification document versioning + Revision discipline (V1/V2/V3 primary + Revision secondary axes).
§11.4.74	Submodule-catalogue-first discovery + extend-don't-reimplement. See <code>submodules-catalogue.md</code> .
§11.4.75	Mechanical enforcement without exception (5-layer git-hook discipline).
§11.4.76	Containers-submodule mandate — use <code>vasic-digital/containers</code> for ALL containerised workloads.
§11.4.77	Regeneration-mechanism-required mandate.
§11.4.78	CodeGraph code-intelligence mandate (npm <code>@colbymchenry/codegraph</code>).

New companion file: `submodules-catalogue.md` — 142 repos categorised into 10 capability groups (Container + lifecycle, AI/LLM + agent, Messaging + observability + storage, Auth + security + middleware, Cross-platform KMP, TypeScript/React + Web, Testing + QA, Apps/tools, Misc/scaffolds).

New file: `QWEN.md` — Qwen Code CLI agent's view of the universal constitution, structured analogously to `AGENTS.md` but tailored to Qwen Code's expectations.

Multi-upstream topology

This repository is hosted on four providers, with GitHub as the primary:

Remote	URL
github (primary)	git@github.com:HelixDevelopment/HelixConstitution.git
gitlab	git@gitlab.com:helixdevelopment1/helixconstitution.git
gitflic	git@gitflic.ru:helixdevelopment/helixconstitution.git
gitverse	git@gitverse.ru:helixdevelopment/HelixConstitution.git

Every commit MUST be pushed to ALL four. The `origin` remote is configured with multiple push URLs so `git push origin <branch>` fans out to every provider in one command.

Recursive inheritance

Between the consuming project's root and any submodule that uses this constitution there may be N intermediate levels. The `find_constitution.sh` helper walks up parents until it locates the constitution submodule, so nested submodules can source the helper without knowing their own depth.

Versioning

Tag the constitution at semantically meaningful checkpoints (`v1.0.0` , `v1.1.0` , `v2.0.0`). Consuming projects pin to a specific tag and upgrade deliberately. Major version bumps mean breaking changes to the universal rules; minor bumps mean additions; patches mean clarifications.

Per the new §11.4.73 spec-versioning discipline (added 2026-05-20), consuming projects that maintain a main specification document follow the **two-axis** rule: primary V for major rewrites, secondary `Revision N` (in metadata table) for additive changes within a primary version.

Contributing

Universal status has to be **earned**, not assumed. Project-specific content does NOT belong here. When in doubt, classify a rule as project-specific and keep it in the consuming project's own Constitution.

A rule is **universal** only if ALL are true:

1. It would be true for at least 3 unrelated projects across different domains.
2. It does not reference a specific service / schema / port / topic / table / vendor / hardware unique to one project.
3. It encodes a *policy* or *principle*, not a *configuration value*.
4. Removing it from any consuming project would be just as wrong as removing it from any other.

A rule is **project-specific** if any are true:

- Mentions a specific service, package, topic, table, ML model, protocol version, hardware, or vendor.
- Encodes a numeric threshold tuned for one codebase (coverage %, latency SLOs, retry counts).
- Describes a workaround for a bug or quirk in a dependency this project uses.

License

See `LICENSE` .